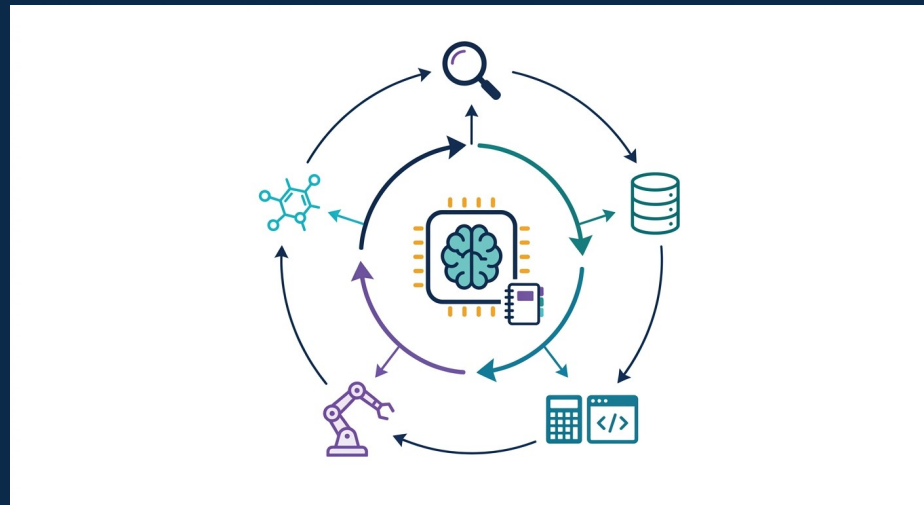




6일차 · 2교시 — 일반 LLM 활용 & 환각 통제

프롬프트 엔지니어링 · API·함수호출 · RAG 파이프라인 · 인용 표준 · 사실성 검증

재직자 3과정 · 6일차(9/12) · 대규모 언어모델과 신약개발 · 이론(심화)

1교시 → 2교시

1교시 (원리·모델)

- LLM 원리·Transformer·정렬
- 범용 LLM ↔ 과학 LM(단백질·화학)
- '무엇이 있나'

2교시 (활용·신뢰)

- 범용 LLM을 연구에 어떻게 쓰나
- 프롬프트·API·RAG
- 환각을 어떻게 통제·검증하나
'어떻게 믿고 쓰나'

+심화

1교시가 '무엇이 있나'였다면 2교시는 '실제로 어떻게 쓰고, 어떻게 믿을 수 있게 만드나'. 신약 현업에서 LLM은 강력하지만 '검증 없는 신뢰'는 위험 — 프롬프트·RAG·체크리스트가 핵심.

2교시에서 볼 것

A. 일반 LLM 활용

프롬프트 엔지니어링(심화)·API·함수호출·구조화 출력

A2. LLM 커스터마이징 4가지

프롬프트 · 미세 튜닝 · 프롬프트 튜닝/LoRA · RAG — 무엇을 언제 (박혜진 커스터마이징 교안)

B. RAG & 환각 통제

환각 원리·유형·완화 · RAG 파이프라인 · 인용·검증

C. 정리

실무 원칙·벤치마크 함정·3교시 예고

+심화

재직자 초점: '그럴듯한 답'을 '검증된 근거'로 바꾸는 실전 기법 — 프롬프트 설계·구조화 출력·RAG·6항목 체크리스트를 손에 익힌다.

PART A

일반 LLM 활용

프롬프트·API·함수호출로 연구를 가속

범용 LLM을 신약연구에 — 어디에 쓰나

작업	LLM 활용
문헌·특허 리뷰	방대한 논문 요약·핵심 추출·비교(RAG 결합)
실험노트·리포트	비정형 기록 구조화·표 추출·요약
가설·아이디어	선행연구 종합→가설·실험 설계 초안
분자·데이터 질의	SMILES 설명·속성 해석·코드 생성(RDKit)
규제·문서 초안	SOP·프로토콜·요약 초안(사람 검수 전제)

쉽게

LLM은 '읽고 · 요약하고 · 초안 쓰고 · 코드 만드는' 만능 조수. 신약연구의 병목인 방대한 문헌 · 비정형 데이터에 특히 유용 — 단, 결과는 반드시 검증(B파트)

재학습 없이 행동을 바꾸는 다섯 개의 손잡이

프롬프트 엔지니어링 주요 기법 5가지

1

16.1

16.2

18

1 역할 부여와 명확한 지시

"당신은 영화 리뷰 감성 분석가입니다.
positive/negative 중 하나만 출력하세요."
역할·작업·출력 형식을 명시

2 제로샷 vs 퓨샷

지시만 주면 제로샷. 입력-출력 예시 몇 개를
보여주면 퓨샷. 형식과 판단 기준을 '
보여주는' 방식

3 사고의 연쇄 (CoT)

"단계별로 생각한 뒤 답하세요" — 중간
추론을 유도해 수학·논리 정확도 향상

4 출력 형식 강제

JSON, 표, 고정 레이블 집합. 스키마 예시를
주면 후처리 파싱 오류가 줄어든다

5 문맥 제공

질문과 함께 관련 자료를 넣어준다. 길거나
자주 바뀌면 검색으로 자동화 → RAG의
출발점

+ 생성 파라미터: temperature



분류는 낮게, 브레인스토밍은 높게. 작은 모델은
높을수록 환각 ↑

▶ 역할 부여 · 제로샷/퓨샷 · 사고의 연쇄(CoT) · 출력 형식 강제 · 문맥 제공 — 재학습 없이 행동만 바꾼다. temperature는 분류·추출은 낮게, 아이디어 발산은 높게(작은 모델일수록 높이면 환각 ↑).

프롬프트 엔지니어링 — 기본기

핵심 알고리즘

1. **역할·맥락** — system: '너는 의약화학 전문가' + 배경 제공
2. **명확한 지시** — 무엇을·형식·제약을 구체적으로
3. **few-shot** — 예시 1~few개로 패턴 제시(in-context)
4. **Chain-of-Thought** — '단계별로 생각' → 추론 정확도 ↑
5. **구조화 출력** — JSON/표 스키마 강제 → 파싱·자동화
6. **검증 유도** — '출처·근거를 함께', '모르면 모른다고'

차별점 (vs 이전)

- ▶ 재학습 없이 행동 제어
- ▶ 형식 고정→파이프라인 연결
- ▶ CoT로 추론 품질 ↑

한계 · 주의

- 프롬프트 민감·불안정
- 여전히 환각 가능

좋은 프롬프트의 6-요소 해부

무엇을 넣나 (구성)

- ① **역할/페르소나**
system: 전문성 · 톤 고정
- ② **맥락/배경**
도메인 · 데이터 · 목적 제공
- ③ **지시(task)**
동사로 명확히 · 범위 한정
- ④ **예시(few-shot)**
입력→출력 패턴 시연

어떻게 통제하나 (제약)

- ⑤ **출력 형식**
JSON 스키마 · 표 · 길이 제한
- ⑥ **제약/가드**
'근거 필수 · 모르면 unknown'
- **우선순위**
system > 예시 > user 지시
- **반복 · 튜닝**
실패 케이스로 프롬프트 개선

+심화

실무 감각: 프롬프트는 '지시문'이 아니라 '사양서(spec)'. 6요소를 빠짐없이 채우면 같은 모델도 재현성 · 정확도가 크게 오른다. 특히 ⑤ · ⑥이 자동화 · 환각 억제 핵심.

프롬프트 — 구조화 출력·근거 요구

의약화학 질의 프롬프트 (개념)

[system] 너는 의약화학 전문가다. 근거 없는 주장 금지.

[user] 다음 표적의 알려진 저해제 계열 3개를 정리.

각 항목: {계열, 대표물질, 근거(PMID/DOI)} # 구조화

출처를 못 찾으면 'unknown'으로. # 환각 억제

[출력] JSON 배열 (파싱→검증 파이프라인)

▶ 핵심: (1) 역할·제약 명시, (2) 스키마 강제, (3) 출처 요구 + '모르면 모른다' — 환각을 줄이고 자동 검증을 가능하게

쉽게

좋은 프롬프트 = '누가·무엇을·어떤 형식으로·어떤 근거로'를 못 박기. 특히 '출처를 붙여라'와 '모르면 모른다고 해라'가 환각을 크게 줄인다

few-shot — 예시로 패턴을 심는다

few-shot: 라벨링 규칙을 예시로 시연

```
[system] 문장에서 (약물, 표적, 관계)를 추출해 JSON으로.  
[예시1] '실데나필은 PDE5A를 저해한다'  
→ {"drug": "sildenafil", "target": "PDE5A", "rel": "inhibits"}  
[예시2] '트라스투주맵은 HER2에 결합한다'  
→ {"drug": "trastuzumab", "target": "HER2", "rel": "binds"}  
[입력] '타다라필은 PDE5A를 억제한다' → ?
```

▶ 예시 2~5개로 '출력 형식+판단 기준'을 동시에 전달 — 규칙을 장황히 쓰는 것보다 예시가 더 정확하고 안정적

+심화

few-shot 실무: (1) 예시는 다양·대표적으로(경계 케이스 포함), (2) 형식을 예시 안에 그대로 보여줌, (3) 예시 순서·개수도 성능에 영향 → 실패 샘플을 예시로 추가하며 튜닝.

추론 강화 — Chain-of-Thought & 분해

핵심 알고리즘

1. **Chain-of-Thought** — '단계별로 풀이를 쓰게' 유도 → 다단계 추론 ↑
2. **분해(decomposition)** — 복잡 질문을 하위 질문으로 쪼개 순차 해결
3. **근거-먼저** — '근거를 먼저 나열 → 그 뒤 결론' 강제
4. **scratchpad** — 중간 계산·가정을 명시적으로 적게
5. **적용 예** — 기전 추론·용량 계산·다단계 문헌 종합

차별점 (vs 이전)

- ▶ 단순 프롬프트 대비 추론 정확도 ↑
- ▶ 중간 과정 노출→검토·디버깅 용이
- ▶ 분해로 오류 국소화

한계 · 주의

- 토큰·비용·지연 ↑
- 쉬운 과제엔 과함
- 과정이 그럴듯해도 결론 오답 가능

안정화 — self-consistency & 자기검토

핵심 알고리즘

1. 여러 번 샘플링 — temperature>0으로 풀이를 N개 생성
2. 다수결(self-consistency) — 가장 많이 나온 최종답 채택 → 분산 ↓
3. 자기검토(critique) — 초안→'오류를 찾아 고쳐라' 2-pass
4. 페르소나 재검토 — '회의적 리뷰어' 시점으로 재평가
5. 교차검증 — 서로 다른 프롬프트/모델 답 대조

차별점 (vs 이전)

- ▶ 단발 답보다 신뢰도 ↑
- ▶ 논리 비약 · 계산오류 감소
- ▶ 자동 파이프라인에 적합

한계 · 주의

- 샘플 수만큼 비용 배수
- 합의가 '틀린 합의'일 수도
- 최종 사실검증은 여전히 필요

구조화 출력 — JSON 스키마로 계약을 건다

출력 스키마 강제 (function/JSON schema 개념)

```
schema = {  
  "type": "object",  
  "properties": {  
    "target": {"type": "string"},  
    "inhibitors": {"type": "array", # 리스트 강제  
      "items": {"required": ["name", "pmid"]}}, # 출처 필수  
    "confidence": {"enum": ["high", "low", "unknown"]}}
```

▶ 스키마를 '계약'으로 강제 → 필드 누락·자유서술 차단, 출처(pmid) 필수화, 불확실성(confidence)까지 구조로 수집

+심화

왜 중요한가: 자유 텍스트는 파싱·검증이 불가능 → 자동화가 막힌다. JSON 스키마(또는 function calling의 파라미터 스키마)로 출력을 '데이터'로 만들면 다음 단계(DB조회·검증)를 코드로 연결할 수 있다.

프롬프트 안티패턴 — 이렇게 하면 틀린다

안티패턴	왜 위험 / 대안
'다 알아서 해줘'(모호)	범위 · 형식 불명→엉뚱한 답 / 지시 · 형식 명시
출처 요구 안 함	환각 · 위조 방치 / '근거 · DOI 필수'
'모르면 모른다' 없음	억지 답 생성 / 불확실성 허용 명시
한 번에 거대 과제	오류 누적 / 분해 · 단계화
출력 자유서술	파싱 · 검증 불가 / JSON 스키마 강제
결과 맹신	검증 생략→오판 / 체크리스트(B파트)

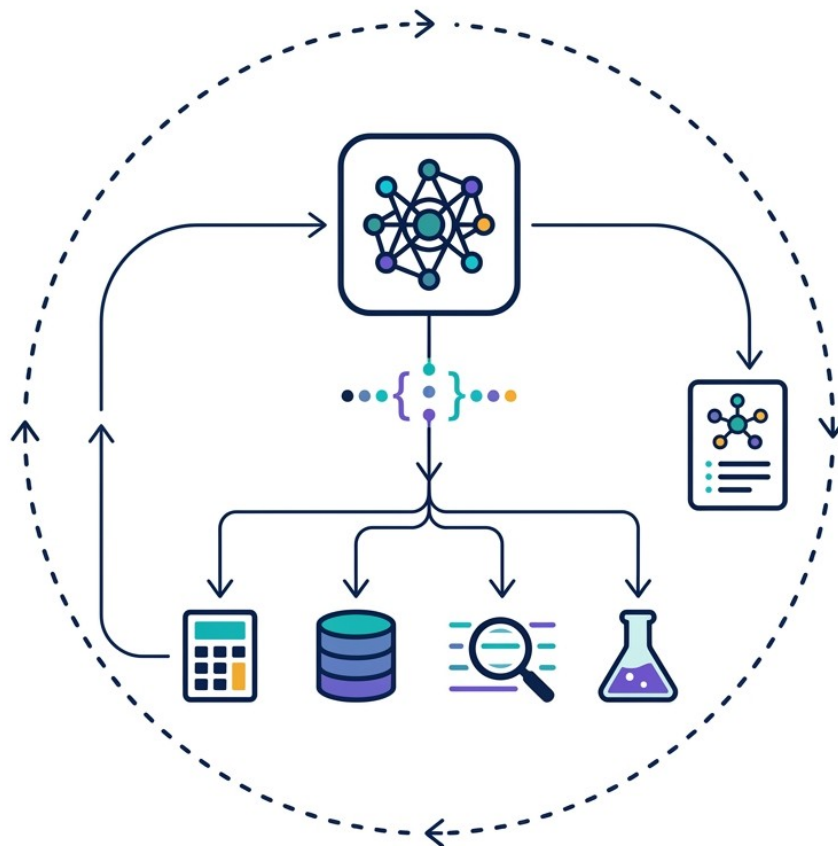


함정

현업 최다 실수: (1) 출처를 안 시키고 (2) 결과를 그대로 믿는 것. 프롬프트에 '근거 필수 · 모르면 모른다'를 넣고, 답은 항상 가설로 취급하라.

함수호출(tool calling) — 모델이 도구를 부른다

무텍스트 개념도: 모델 → 구조화 JSON 호출 → 외부 도구(계산·DB·검색·화학) → 결과 회신 루프



+심화

모델이 스스로 '어떤 도구를, 어떤 인자로' 호출할지 구조화(JSON)해 내보내면, 시스템이 실제 도구(계산기·DB·검색·RDKit)를 실행하고 결과를 다시 모델에 돌려준다. 이 요청-응답 루프가 3교시 '에이전트'의 토대.

API · 함수호출 — 연구 파이프라인에 연결

API 활용

- 프로그램에서 LLM 호출(배치·자동화)
- temperature·top_p로 다양성 제어
- system/tools/messages 구성
비용=토큰(3교시 실습)

구조화·함수호출

- JSON 스키마 강제 출력
- function/tool calling → 외부 도구 호출
- 에이전트의 토대(3교시)
RDKit·DB·검색 연결

쉽게

챗창을 넘어 '코드로' LLM을 부르면 대량 자동화가 된다. 출력을 JSON으로 고정하고, 함수호출로 외부 도구(계산·검색)를 붙이면 → 에이전트(3교시)로 진화

함수호출 — 도구 스키마 정의(개념)

tool 정의 + 모델의 호출 → 실행 → 회신

```
tools = [{  
    "name": "pubmed_search",          # 도구 이름  
    "description": "질의어로 PubMed를 검색",  
    "parameters": {"query": "string", "top_k": "int"}}]  
# 모델 출력(구조화):  
call = {"tool": "pubmed_search",  
        "args": {"query": "PDE5 inhibitor resistance", "top_k": 5}}  
# 시스템이 실제 검색 실행 → 결과를 모델에 회신 → 최종답 생성
```

▶ 모델은 '무엇을 검색할지'만 정하고, 실제 실행은 우리 코드가 담당 → 근거 있는 답 + 재현 가능한 행동 로그

+심화

핵심: 도구 스키마를 잘 쓰면 LLM이 계산·검색·시뮬레이션을 '적절한 때' 호출한다. 단, 어떤 도구를 언제 부를지는 모델 판단이라 오호출 가능 → 인자 검증·권한 제한 필수(4교시 보안).

출력 제어 — temperature · top_p · seed

다양성 파라미터

- **temperature**
0=결정적, ↑=창의·발산
- **top_p (nucleus)**
확률 상위 집합만 샘플링
- **사실질의·추출**
temperature 낮게(0~0.3)
- **아이디어 발산**
temperature 높게(0.7~1.0)

재현·안정

- **seed 고정**
같은 입력→같은 출력(가능 모델)
- **max_tokens**
길이·비용 상한
- **stop 시퀀스**
형식 이탈 방지
- **로그 보관**
프롬프트·파라미터·버전 기록

쉽게

temperature = '모험 다이얼'. 사실을 뽑을 땐 낮춰 안정적으로, 브레인스토밍엔 높여 다양하게. self-consistency는 일부러 높여 여러 답을 뽑아 다수결하는 것.

실무 — 모델 선택과 비용 · 보안

모델 선택 기준

- 과제 난도 ↔ 모델 급(비용) 매칭
- 긴 문서 → 긴 컨텍스트 모델
- 정형 · 대량 → 저가 · 고속 모델
불필요한 고급모델 낭비 주의

비용 · 보안 실무

- 비용 = 토큰(입력 + 출력) · 캐싱 활용
- 민감데이터 → 온프레임/계약 · 마스킹
- 모델버전 고정 · 로그 보관
재현 · 감사(4교시)

+심화

실무 팁: 쉬운 작업엔 싸고 빠른 모델, 어려운 추론엔 고급 모델로 '급을 나눠' 쓰면 비용을 크게 아낀다. 2026 현행 프론티어=GPT-5 계열 · Claude Opus 5/Sonnet 5 · Gemini 3 · 오픈 Llama 4(세부 버전 2026.9 확인 권장). 민감정보(미공개 타겟 · 환자데이터)는 반드시 처리 정책 확인(4교시 보안).

비용 구조 — 토큰 · 캐싱 · 컨텍스트

항목	의미 / 실무 절감법
입력 토큰	프롬프트 · 문서 길이 → 프롬프트 다이어트 · 요약 선행
출력 토큰	생성 길이 → max_tokens · 간결 형식 지정
프롬프트 캐싱	반복 system · 문서 재사용 → 캐시로 입력비 ↓
컨텍스트 한계	길수록 비용 · '중간 소실' ↑ → 필요한 RAG로
배치 · 모델급	저난도는 소형 · 배치 → 단가 ↓



함정

'긴 컨텍스트에 다 넣기'는 비싸고 위험 — 중간 정보 소실(lost-in-the-middle) · 비용 급증. 필요한 근거만 RAG로 골라 넣는 것이 싸고 정확하다.

민감데이터 — 넣기 전에 확인할 것

위험

- 미공개 타겟 · 후보구조 유출
- 환자 · 임상 개인정보(규제 대상)
- 계약 · NDA 위반 가능
- 프롬프트 인젝션(4교시)
외부 문서에 숨은 지시

실무 가드

- 전송 전 마스킹 · 비식별화
- 온프레임/기업계약 · 데이터 미학습 옵션
- 접근권한 · 로그 · 감사 추적
- 사내 정책 · IRB 확인
의심되면 넣지 않기

+심화

원칙: '이 데이터가 외부로 나가도 되는가'를 넣기 전에 판단. 미공개 IP · 환자정보는 기본적으로 외부 API 금지 또는 계약 · 비식별화 후. 상세 위험 · 방어는 4교시(보안).

후보물질 생성·평가에 LLM

- SMILES/서열을 프롬프트로 — 설명·유사물질·변형 제안
- 화학 LM(1교시)·생성모델과 결합해 아이디어 발산
- 물성·ADMET 도구 호출로 후보 스크리닝 자동화(3교시 에이전트)
- 단, LLM 단독 분자 '생성'은 validity·합성가능성 취약 → 전용 생성모델 병용

⚠ **함정**

LLM에게 직접 'SMILES 만들어줘' 하면 문법 오류·비현실 분자가 흔하다. 후보 생성은 전용 생성모델(REINVENT 등, 4일차)에 맡기고 LLM은 오케스트레이션·해석에 쓰는 게 안전

멀티모달 LLM — 텍스트+분자+구조

1. 멀티모달 LLM의 등장:

멀티모달 LLM은 텍스트뿐만 아니라 이미지, 분자 구조, 생물학적 시퀀스 등의 다양한 데이터 형태를 처리할 수 있습니다. 이를 통해 약물-타겟 상호작용, 단백질-단백질 상호작용 등 복잡한 생물학적 과정을 이해하는 데 활용됩니다. 예를 들어, AI-powered drug discovery 플랫폼들은 텍스트 및 분자 구조 데이터를 함께 처리하여 약물의 효과와 부작용을 예측하는 모델을 개발하고 있습니다.

2. 신약 재창출(Drug Repurposing)에의 활용:

LLM은 기존 약물의 새로운 적응증을 발견하는 데에도 활용되고 있습니다. 최근 연구에서는 LLM을 통해 COVID-19 치료제로 사용될 수 있는 기존 약물 후보를 빠르게 식별한 사례가 있습니다.

3. 약물-타겟 상호작용 예측:

LLM은 자연어로 기술된 약물 및 타겟 정보에서 상호작용을 파악하고 예측하는 데 사용됩니다. 이를 통해 새로운 약물 후보를 효율적으로 식별할 수 있습니다.

▶ LLM은 텍스트를 넘어 분자(SMILES)·단백질 구조·이미지를 한 모델에서 함께 다룬다. 화학 멀티모달(예: ChemMLLM 2026, 확인 권장)이 대표 — 1교시 과학 LM과 연결.

PART A2

LLM 커스터마이징 — 4가지 방법

프롬프트 엔지니어링 · 미세 튜닝 · 프롬프트 튜닝 · RAG — 무엇을 언제 쓰나

프롬프트만으로 안 되는 네 가지 — 그래서 무엇이 필요한가

1

16.1

16.2

18

프롬프트 엔지니어링의 한계 — 왜 다음 단계가 필요한가

지식의 한계

모델이 모르는 사실은 아무리 잘 물어봐도 나오지 않는다

RAG (18장)

문맥 창고의 한계

프롬프트 길이에 상한. 길수록 비용 ↑, 정확도 ↓

RAG의 검색으로 필요한 부분만 선별

일관성·정확도의 한계

예시 몇 개로는 도메인 특화 정확도를 끝까지 못 올림. 프롬프트가 조금만 바뀌어도 흔들림

미세 튜닝 (16.1) / 프롬프트 튜닝 (16.2)

추론 비용

긴 퓨샷 프롬프트는 매 호출마다 토큰 비용 발생

학습으로 모델에 '내재화'

프롬프트 엔지니어링 = 말을 잘 거는 기술 / 튜닝 = 모델이 말을 잘 알아듣게 바꾸는 기술 / RAG = 모델에게 자료를 쥐여주는 기술

▶ 지식의 한계→RAG, 문맥 창·비용의 한계→검색으로 선별, 일관성·정확도의 한계→튜닝. 한 문장 요약: 프롬프트=말을 잘 거는 기술 / 튜닝=말을 잘 알아듣게 바꾸는 기술 / RAG=자료를 쥐여주는 기술.

무엇이 바뀌고 훈련이 얼마나 드나 — 네 방법 대조

네 가지 접근법 한눈에 보기

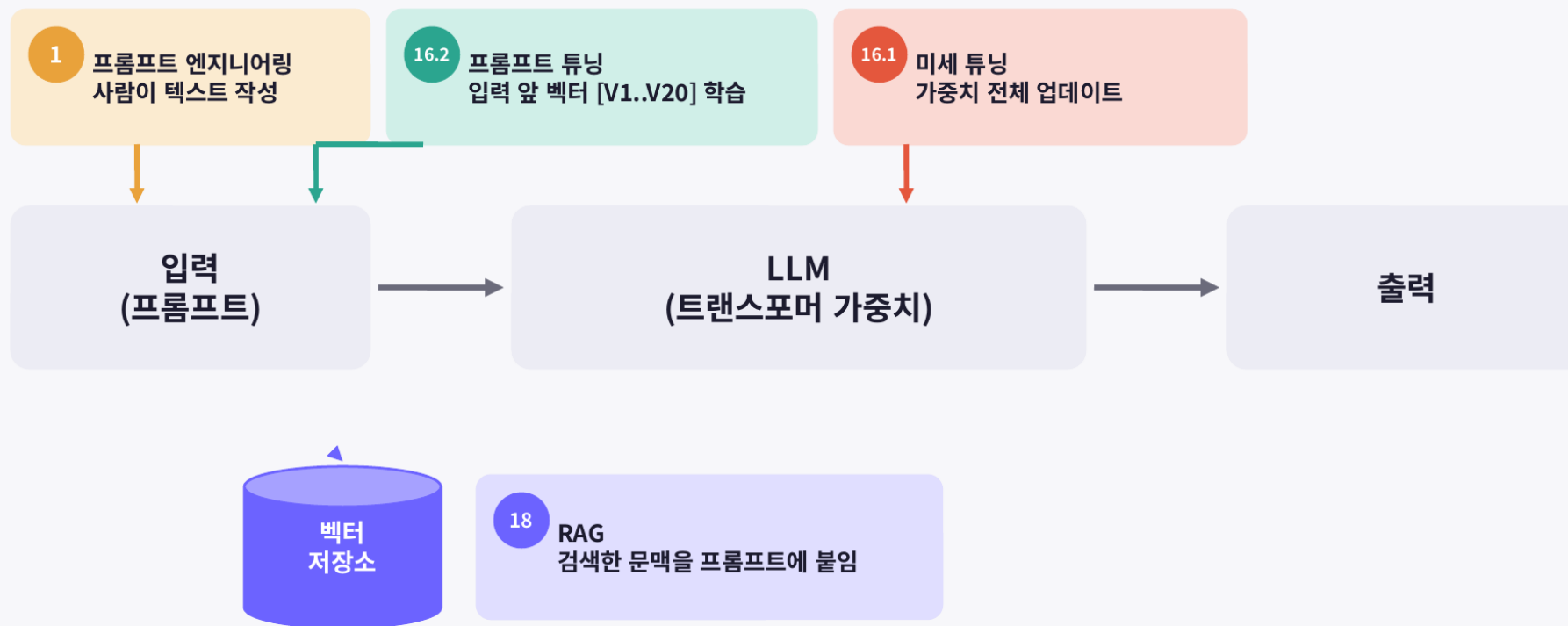
	1. 프롬프트 엔지니어링	16.1 미세 튜닝	16.2 프롬프트 튜닝	18. RAG
핵심 아이디어	사람이 입력 텍스트를 설계	모델 가중치 자체를 업데이트	가중치 동결, 입력 앞 소프트 프롬프트 벡터만 학습	모델은 그대로, 검색한 문맥을 프롬프트에 추가
무엇이 바뀌나	프롬프트 문자열	모델 전체 (수백 MB~GB)	프롬프트 임베딩 (61KB)	없음 (벡터 DB만 추가)
훈련 필요	없음	있음 (무거움)	있음 (가벼움)	없음
데이터 요구량	예시 몇 개	25,000 샘플	5,000 샘플	원문 문서 (PDF)
적합한 상황	출력 형식·말투·역할 지정, 빠른 실험	특정 작업/도메인 정확도를 최대로	한 베이스 모델로 여러 작업 전환	모델이 모르는 최신·사내·비공개 데이터로 답변
실습	(개념)	BERT + IMDb, Trainer	BERT + IMDb, 직접 훈련 루프	소셜 PDF + ChromaDB + Ollama/GPT

▶ 훈련 필요 여부(없음/무거움/가벼움/없음)와 '무엇이 바뀌나'(프롬프트 문자열/모델 전체/프롬프트 임베딩/벡터 DB)가 네 방법을 가르는 두 축. 데이터 요구량도 25,000샘플 vs 5,000샘플 vs 원문 PDF로 크게 다르다.

파이프라인 어디를 건드리나 — 가중치를 바꾸는 건 하나뿐

각 방법이 손대는 곳이 다르다

입력 → 모델 → 출력 파이프라인에서 어디를 바꾸는가



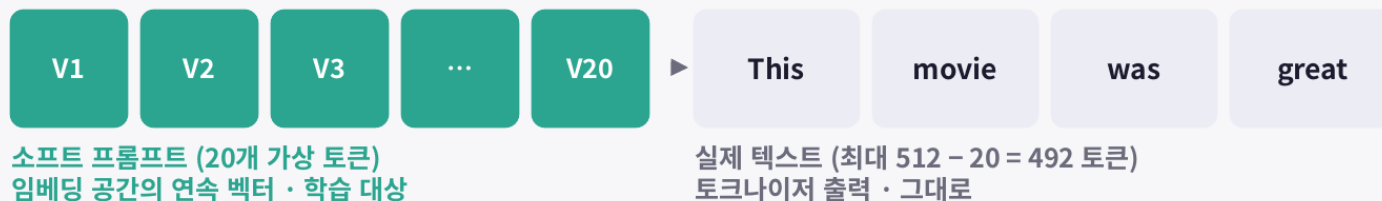
모델 가중치를 바꾸는 것은 미세 튜닝뿐. 나머지 셋은 모델을 '그대로' 둔다.

▶ 모델 가중치를 바꾸는 것은 미세 튜닝뿐. 프롬프트 엔지니어링 · 프롬프트 튜닝은 '입력 앞', RAG는 '검색한 문맥을 프롬프트에 붙임' — 나머지 셋은 모델을 그대로 둔다.

모델은 동결, 가상 토큰 20개만 학습한다

소프트 프롬프트: 입력 앞에 붙는 '학습된 지시문'

모델에 실제로 들어가는 입력



BERT (bert-base-uncased) — requires_grad_(False) 동결

분류 헤드 → 긍정 / 부정

왜 유용한가

- 효율성: 작업마다 모델 전체가 아니라 프롬프트 벡터만 저장 (61KB)
- 재사용: 한 베이스 모델 + 여러 프롬프트 파일 = 여러 작업
- 대규모 모델에서는 전체 미세 튜닝 성능과 일치하거나 초과

▶ BERT를 통째로 동결(requires_grad_(False))하고, 입력 앞 20개 가상 토큰 벡터만 학습. 작업마다 모델 전체가 아니라 프롬프트 벡터(61KB)만 저장하면 되므로 한 베이스 모델로 여러 작업을 돌릴 수 있다.

'프롬프트 엔지니어링'과 '프롬프트 튜닝'은 다른 것

1

16.1

16.2

18

하드 프롬프트 vs 소프트 프롬프트

1장의 '프롬프트 엔지니어링'과 16.2의 '프롬프트 튜닝'은 이름만 비슷하다

1

하드 프롬프트 (프롬프트 엔지니어링)

```
"Classify the sentiment:
This movie was great"
```

- 이산적인 텍스트 토큰 — 어휘 사전에 있는 단어
- 사람이 작성하고 읽을 수 있다
- 훈련 없음, 즉시 실험
- 모델은 전혀 바뀌지 않음

16.2

소프트 프롬프트 (프롬프트 튜닝)

```
[V1][V2]...[V20] This movie was great
# V = 768차원 실수 벡터
```

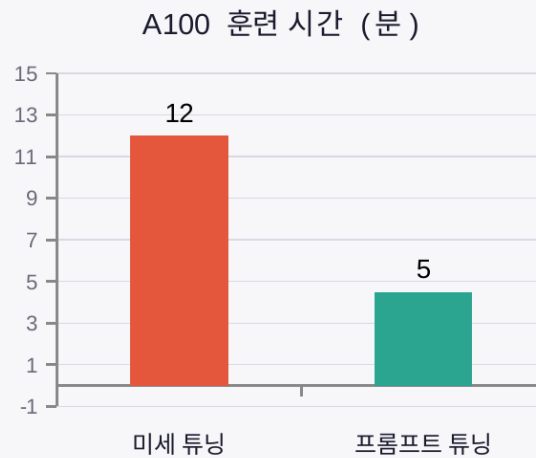
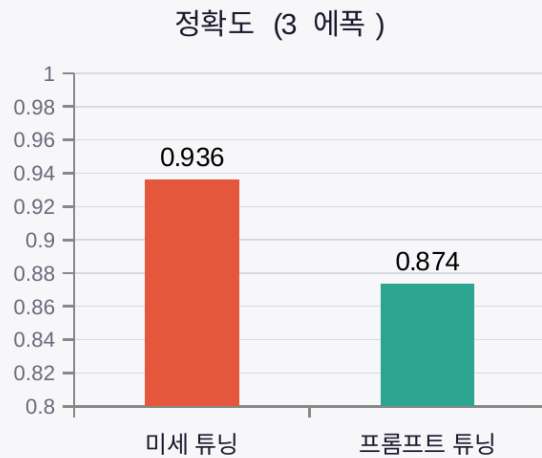
- 모델 임베딩 공간에 존재하는 연속 벡터
- 사람이 읽을 수 없다 — 어휘 사전에 없는 '단어'
- 경사 하강법으로 학습 (모델은 동결)
- nn.Parameter로 등록된 유일한 학습 파라미터

소프트 프롬프트는 '모델의 동작을 안내하는 학습된 지시문'

▶ 하드=사람이 읽고 쓰는 실제 텍스트 토큰(훈련 0). 소프트=어휘 사전에 없는 연속 벡터로, 경사하강법으로 학습되는 유일한 파라미터. '프롬프트 엔지니어링'과 '프롬프트 튜닝'을 혼동하지 말 것.

정확도 6%p를 위해 7,000배의 파라미터를 쓸 것인가

같은 과제, 다른 비용: 미세 튜닝 vs 프롬프트 튜닝



학습 파라미터 수

~110M

미세 튜닝

~15K

프롬프트 튜닝

약 7,000분의 1

20 토큰 × 768 차원 = 15,360개
막대 그래프로는 보이지 않는 차이

	미세 튜닝 (16.1)	프롬프트 튜닝 (16.2)
훈련 대상	BERT 전체 + 분류 헤드 (~110M)	소프트 프롬프트 20 × 768 (~15K)
훈련 데이터 / 학습률	25,000 샘플 / 2e-5	5,000 샘플 / 1e-2
훈련 루프	Trainer가 캡슐화	직접 작성
저장 크기	수백 MB (모델 전체)	61 KB (프롬프트만)

▶ 동일 IMDb 감성분류에서 정확도는 0.936 vs 0.874, A100 훈련 12분 vs 5분, 학습 파라미터는 ~110M vs ~15K(약 7,000분의 1), 저장은 수백 MB vs 61KB. '정확도 6%p'를 이 비용 차이와 견주어 판단한다.

학습 파라미터를 모델 바깥에 둘까, 안쪽에 둘까(LoRA)

1

16.1

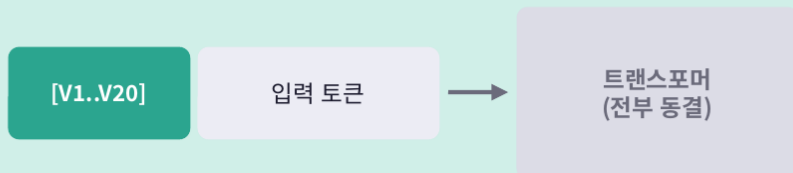
16.2

18

[보충] PEFT 안에서의 위치: 프롬프트 튜닝 vs LoRA

둘 다 베이스 가중치는 동결. 차이는 '학습 파라미터를 어디에 두느냐'

프롬프트 튜닝 — 모델 바깥(입력)



- 내부 층을 전혀 건드리지 않음, 정방향 경로 동일
- 파라미터 = 토큰 수 $\times$ 임베딩 차원 $\approx 15K$
- 시퀀스가 가상 토큰 수만큼 길어짐
- 큰 모델(10B+)일수록 미세 튜닝에 근접

LoRA — 모델 안쪽(가중치 옆)



- 어텐션 Q, V 프로젝트 옆에 저랭크 행렬 삽입
- 파라미터 = 원본의 0.1~1% (수십만~수백만)
- 학습 후 $W' = W + BA$ 로 병합 $\rightarrow$ 추론 비용 0
- 모델 크기와 무관하게 안정적, 현재 사실상 표준

	프롬프트 튜닝	LoRA
표현력	제한적 (BERT급에선 격차 큼)	미세 튜닝에 근접
멀티태스크 서빙	프롬프트 파일 교체, 배치 내 혼합 가능	어댑터 교체 (병합 안 하면)
초기화 민감도	높음	낮음 (B=0이면 시작점 = 원본)

▶ 둘 다 베이스 가중치는 동결. 차이는 '학습 파라미터를 어디에 두느냐' — 프롬프트 튜닝은 모델 바깥(입력), LoRA는 모델 안쪽(어텐션 Q·V 프로젝트 옆 저랭크 행렬). 1교시에서 본 LoRA가 여기에 놓인다.

네 방법을 신약개발에서 언제 쓰나

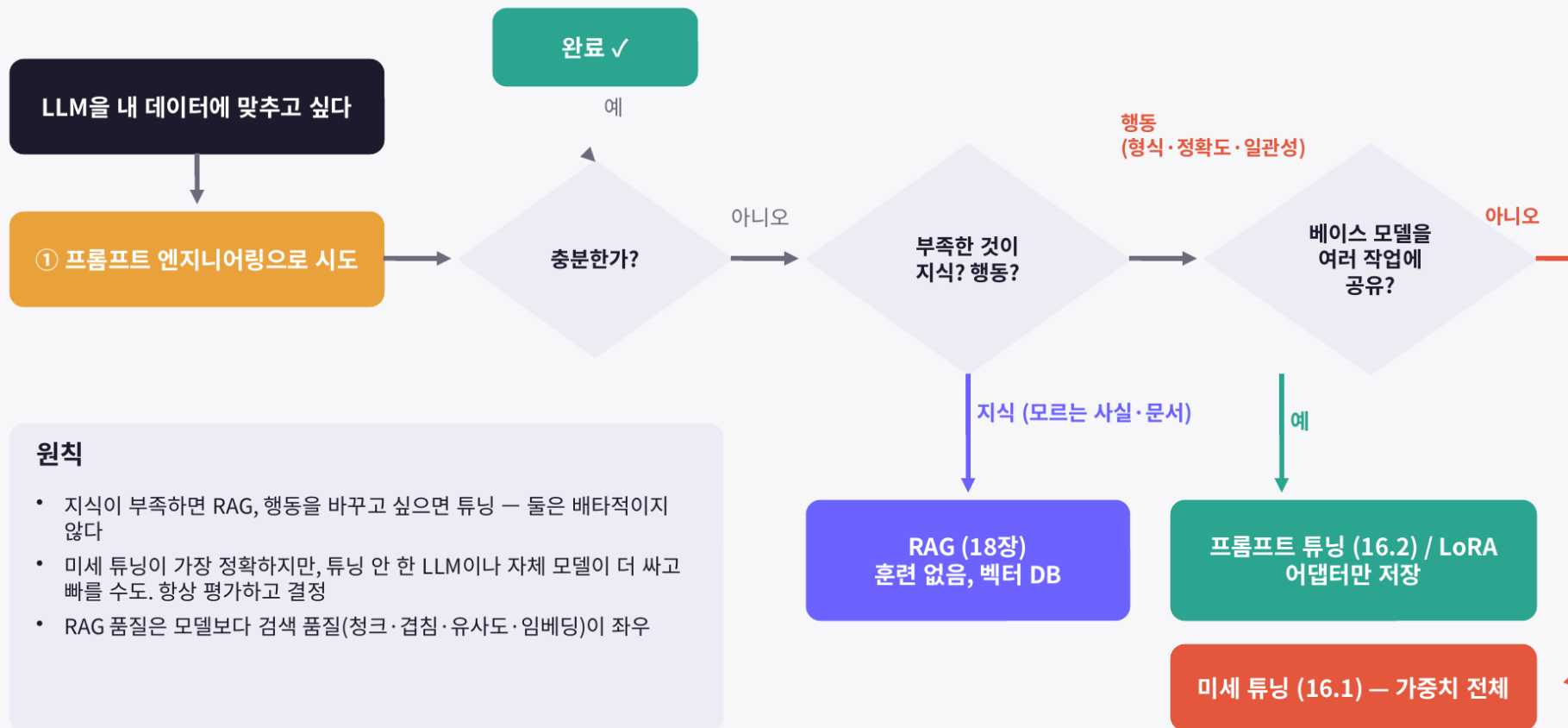
방법	신약개발에서 쓰는 순간	현실적 조건 / 6일차 연결
프롬프트 엔지니어링	논문 요약·표 추출·가설 브레인스토밍 — 지금 당장, 훈련 없이 시도	가장 먼저 시도. 2교시 앞부분 · nb1 시스템 메시지로 재사용
RAG	최신 논문·자사 실험노트·비공개 규제문서를 근거로 답하게 — 출처 추적 필수일 때	모델은 그대로, 벡터 DB만 추가. 6일차 4절 RAG 실습
프롬프트 튜닝 / LoRA	한 베이스 모델로 ADMET 분류·문헌 트리아지 등 여러 과제를 경량 전환	어댑터·프롬프트 파일만 교체. 라벨 수천 건 규모부터
미세 튜닝	사내 도메인 코퍼스(합성 경로 노트·독성 리포트)로 정확도를 끝까지 올릴 때	라벨 수만 건 + GPU + 모델 사본 관리 비용. 마지막 카드

+심화

판단 순서: ①프롬프트로 먼저 → ②부족한 것이 '지식'이면 RAG, '행동(형식·정확도·일관성)'이면 튜닝 → ③여러 작업 공유면 프롬프트 튜닝/LoRA, 한 작업 정확도 최우선이면 미세 튜닝. 오늘 실습(4절 RAG, nb1 에이전트)은 ①·②를 손에 익히는 단계.

지식이 부족한가, 행동이 부족한가 — 선택 흐름도

어떤 방법을 고를 것인가: 의사결정 가이드



▶ 지식이 부족하면 RAG, 행동을 바꾸고 싶으면 튜닝 — 둘은 배타적이지 않다. RAG 품질은 모델보다 검색 품질(청크·겹침·유사도·임베딩)이 좌우한다는 점이 다음 PART B의 출발점.

네 숫자만 기억한다 — 훈련 0 · 93.6% · 61KB · 1000/200

핵심 메시지: 기억할 숫자 하나씩

1 프롬프트
엔지니어링

훈련 0

입력 텍스트를 설계. 항상 첫 시도. 여기서 만든 시스템 메시지는 RAG에서 재사용

16.1 미세 튜닝

93.6%

가중치를 직접 업데이트. A100 12분, 3 에폭. 정확도 최고, 모델 복사본 수백 MB

16.2 프롬프트 튜닝

61 KB

가중치 동결, 입력 앞 벡터만 학습. 정확도 0.87, 프롬프트 파일 교체로 멀티태스킹

18 RAG

1000 / 200

훈련 대신 검색으로 문맥 주입. `chunk_size / overlap`. 모델이 모르는 것을 답하게

▶ 프롬프트=훈련 0 / 미세 튜닝=93.6% / 프롬프트 튜닝=61KB / RAG=1000·200(chunk_size·overlap). 네 숫자만 기억해도 회의에서 방법 선택 논의가 가능하다.

PART B

RAG & 환각 통제

'그럴듯한 오답'을 근거로 붙잡는다

LLM의 한계 — 환각·프롬프트 주입

프롬프트 주입 (Prompt Injection)

악의적인 프롬프트(질문)를 입력하여 LLM이 본래 정책이나 가이드라인에 구애받지 않고 공격자 의도대로 작동하도록 하는 취약점이다. LLM 접근 권한 제어 강화와 외부 콘텐츠 분리 등을 통해 완화할 수 있다.

불완전한 출력 처리 (Insecure Output Handling)

LLM에서 생성된 출력이 충분한 검증 과정 없이 다른 시스템으로 전달될 경우, 원격 코드 실행 등의 위협이 발생할 수 있다. 제로 트러스트 접근 방식 사용과 입력 유효성 검사 등을 통해 예방할 수 있다.

학습 데이터 중독 (Training Data Poisoning)

사전 학습 데이터를 조작하여 모델의 보안성과 효율성을 손상시키는 취약점이다. 사용자가 오염된 정보에 노출되거나 시스템 성능 저하를 초래할 수 있다. 안정성이 검증된 학습 데이터를 사용하여 예방할 수 있다.

모델 서비스 거부 (Model Denial of Service)

공격자가 대량의 리소스를 소모시켜 다른 사용자의 서비스 품질을 저하시키고 높은 리소스 비용을 발생시킨다. 사용자 입력 제한 규칙 준수와 리소스 사용량 제한 등을 통해 예방할 수 있다.

공급망 취약점 (Supply Chain Vulnerabilities)

체계적인 방식이나 도구 없이 LLM 공급망을 관리하기 어렵기 때문에, 소프트웨어 공급망 취약점과 유사한 위협이 발생할 수 있다. 신뢰할 수 있는 공급 업체 사용과 패치 정책 구현 등을 고려해야 한다.

민감 정보 노출 (Sensitive Information Disclosure)

LLM의 답변을 통해 민감한 정보가 노출될 수 있으며, 이로 인해 개인 정보 침해나 지적 재산에 대한 무단 액세스가 발생할 수 있다. 적절한 데이터 정제 기술을 사용하여 민감 데이터가 학습 데이터에 포함되지 않도록 해야 한다.

불완전 플러그인 설계 (Insecure Plugin Design)

LLM 플러그인은 사용자가 다른 애플리케이션 사용 중 자동으로 호출되는 확장 기능이다. 모델이 다른 플랫폼에서 제공될 경우 애플리케이션 실행을 제어할 수 없으므로 원격 코드 실행 등의 위협이 발생할 수 있다. 민감한 작업 실행 시 수동 승인을 요구하고 인증 ID를 적용하는 등의 방법으로 예방할 수 있다.

과도한 에이전시 (Excessive Agency)

기능 호출 권한을 가진 에이전트가 LLM의 출력에 대응하여 해로운 작업을 수행할 수 있다. 세분화된 기능을 갖춘 플러그인을 사용하고 최소한의 권한으로 제한하는 등의 방법으로 예방할 수 있다.

과도한 의존 (Overreliance)

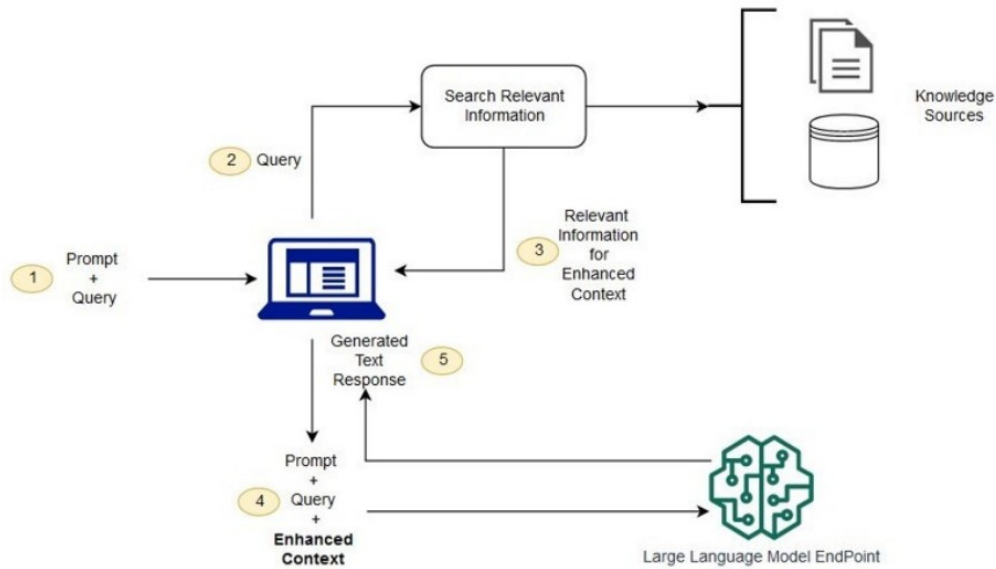
LLM이 사실과 다른 정보나 부적절한 콘텐츠를 생성하는 환각 현상이 발생할 수 있다. 이러한 결과를 검토하지 않고 신뢰하면 잘못된 정보가 전달될 수 있다. 파인튜닝, 임베딩, RAG 기법 등을 활용하여 출력 품질을 개선하고, 전문가 검증을 받으며, 사용자가 LLM의 한계를 명확히 인식하도록 하는 방법으로 예방할 수 있다.

모델 도난 (Model Theft)

공격자가 해킹을 통해 LLM 모델에 무단으로 접근하거나 모델이 유출될 수 있다. 강력한 보안 조치를 통해 예방할 수 있다.

▶ LLM은 학습에 없는 것을 '그럴듯하게' 지어내고(환각), 외부 문서에 숨은 지시에 휘둘릴 수 있다(프롬프트 주입) — 근거 부착(RAG)과 검증이 필요한 이유.

RAG — 검색 증강 생성 구조

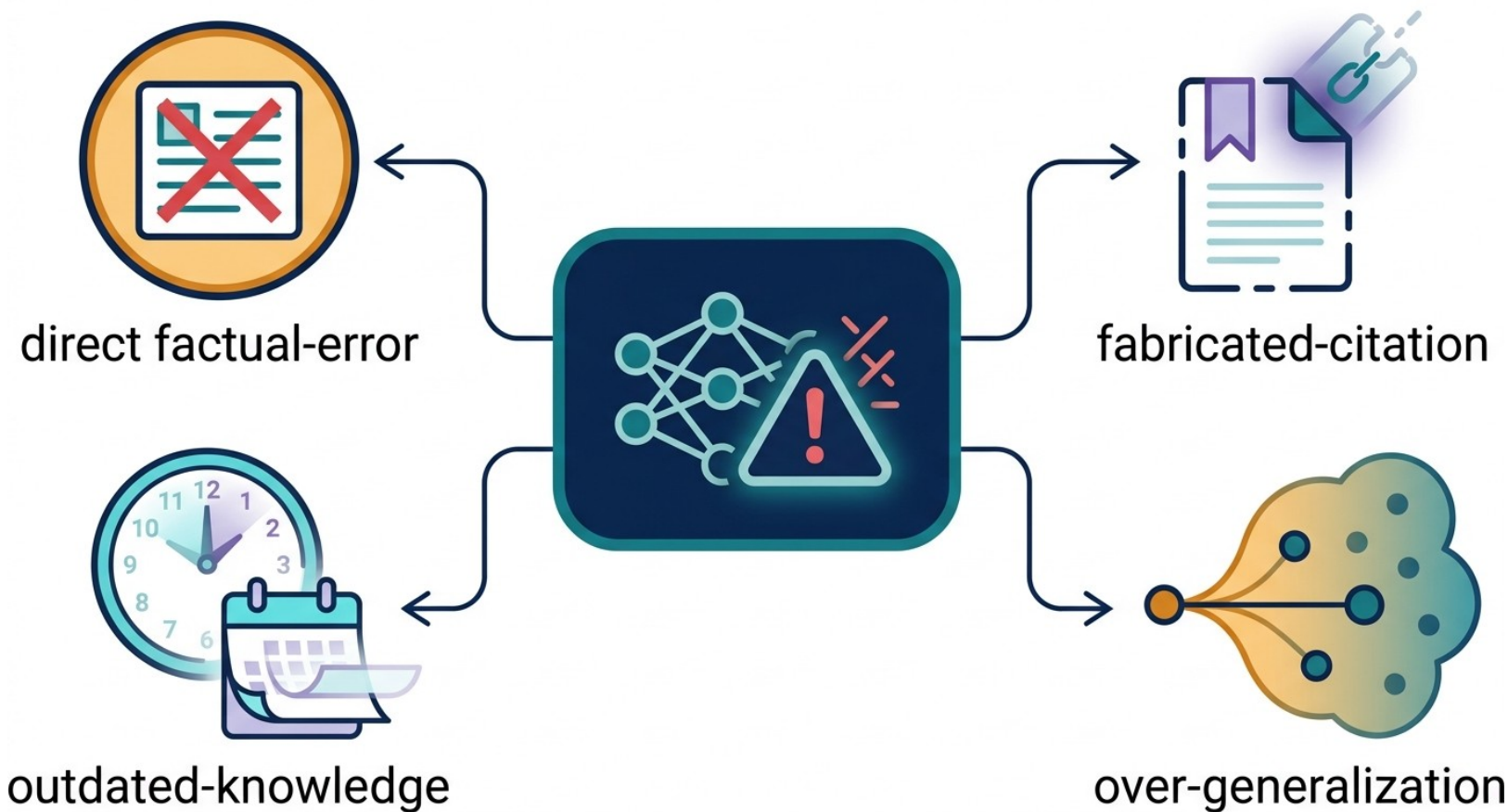


- 외부데이터생성
- 관련정보 검색
- LLM 프롬프트 확장
- 외부데이터 업데이트

▶ 외부 데이터→검색(retrieve)→관련 근거를 프롬프트에 확장(augment)→생성(generate). '오픈북 시험'처럼 실제 문서에 근거해 답한다.

환각의 네 갈래 — 무엇을 조심하나

개념도(영문 라벨): 사실오류(factual-error) · 출처위조(fabricated-citation) · 최신성 결여(outdated) · 과잉일반화(over-generalization)가 한 모델에서 뻗어나온다



쉽게

환각은 한 종류가 아니다: 기전·수치를 틀리는 사실오류, 없는 논문·가짜 DOI를 만드는 출처위조, 철회·구정보를 최신처럼 내는 최신성 결여, 좁은 근거를 확정처럼 말하는 과잉일반화. 유형별로 방어법이 다르다.

왜 환각(hallucination)이 생기나

- LLM = 확률적 '다음 토큰 이어쓰기' — 사실 저장소가 아님
- 학습에 없거나 애매하면 '가장 그럴듯한' 텍스트를 생성(사실 여부 무관)
- 결과: 없는 논문·가짜 DOI·틀린 수치를 유창하게 제시
- 학습 시점 고정 → 최신 정보 결여도 오답 원인

쉽게

환각 = '아는 척 이어쓰기'. 모델은 '맞는 말'이 아니라 '그럴듯한 말'을 만들도록 학습됐다. 그래서 유창할수록 더 속기 쉽다(Galactica 교훈, 1교시)

환각의 원리 — 왜 유창할수록 위험한가

핵심 알고리즘

1. 목적함수 — '다음 토큰 확률'을 맞추도록 학습(사실이 아님)
2. 분포 채우기 — 근거 없어도 가장 그럴듯한 토큰을 채워넣음
3. 유창성 편향 — 문법·톤이 자연스러워 사실처럼 들림
4. 보상·아부 — 사람 선호 학습→'듣기 좋은 답' 경향(sycophancy)
5. 지식 컷오프 — 학습 이후 정보 부재 → 낡은 답

차별점 (vs 이전)

- ▶ '유창함 ≠ 정확성'을 이해
- ▶ 왜 출처·검증이 필수인지 설명
- ▶ 완화 지점을 특정 가능

한계 · 주의

- 모델 내부만으론 사실성 보장 불가
- '자신 있게 틀림'이 가장 위험
- 프롬프트만으론 근절 불가→RAG

환각 유형 — 신약연구 맥락

유형	예시 / 신약 맥락
사실 오류(intrinsic)	기전·수치·관계를 틀리게 서술
출처 위조(fabrication)	존재하지 않는 논문·가짜 DOI/PMID
최신성 결여	철회·갱신된 정보를 최신처럼
과잉 일반화	제한적 근거를 확정처럼 진술
수치 붕괴	임상 수치·단위·통계를 왜곡



함정

특히 위험: 가짜 DOI/PMID를 '진짜처럼' 붙이는 출처 위조. 전공자도 속기 쉬움 — 인용은 반드시 실재 검증(아래 체크리스트)

환각 완화 — 4개 층으로 방어

층위	기법 / 효과
① 프롬프트	'근거 필수·모르면 unknown'·CoT·자기검토
② 근거 주입(RAG)	외부 문헌·DB 검색→grounding(가장 강력)
③ 디코딩·양상블	self-consistency·낮은 temperature
④ 사후 검증	출처 실재 조회·원문 대조·인간 검토
결합 원칙	단일 기법 아님 — 층을 겹칠수록 견고

+심화

환각은 '없앤다'가 아니라 '층층이 줄인다'. 프롬프트로 억제→RAG로 근거 부착→양상블로 안정화→사후 검증으로 최종 확인. 이 4층이 재직자 실무의 표준 방어선.

환각을 '탐지'하기 — 불확실성 신호

모델 쪽 신호

- **자기 확신 표현**
'high/low/unknown'을 출력에 강제
- **일관성 검사**
여러 샘플이 갈리면 불확실
- **근거 부재**
RAG에서 근거 못 찾음=위험
- **보정(calibration)**
확신도와 실제 정답률 대조

운영 쪽 대응

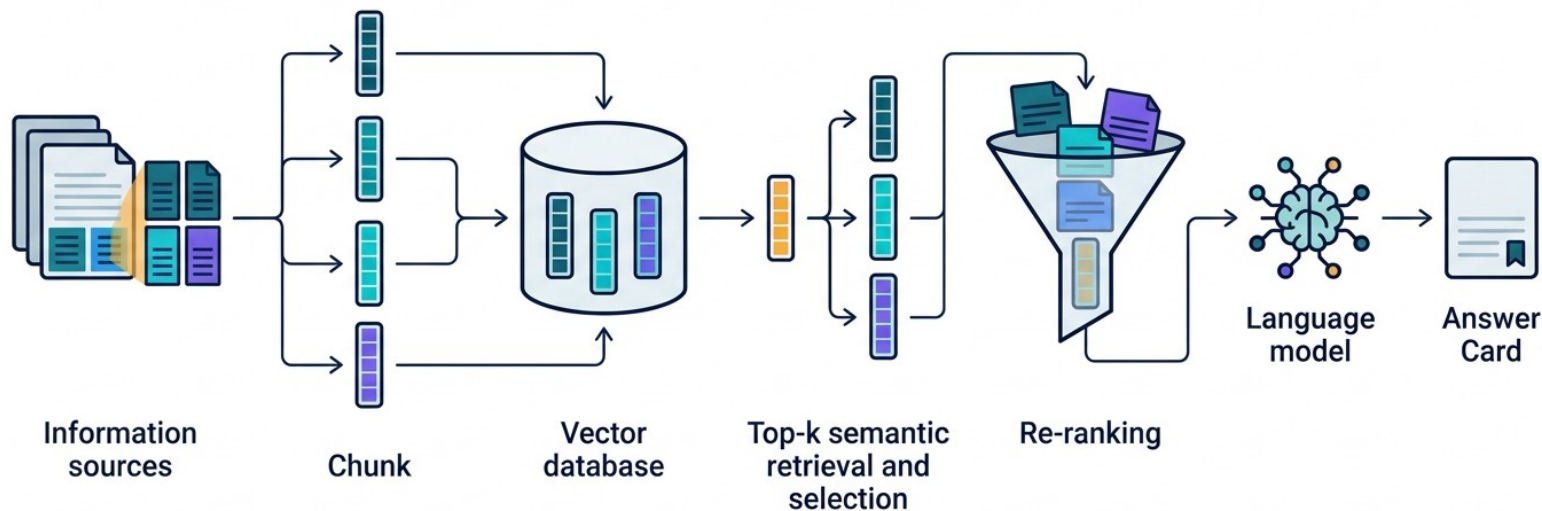
- **저확신→보류/사람**
불확실 항목만 사람 검토
- **근거 없으면 unknown**
억지 답 대신 명시적 유보
- **flag & 추적**
의심 답을 로그·재검
- **임계값 운영**
위험 도메인은 보수적 컷

+심화

핵심: 환각을 0으로 만들 수 없다면 '어디가 위험한지'를 신호로 잡아 사람에게 넘긴다. 여러 샘플의 불일치·근거 부재·낮은 자기확신은 유용한 위험 신호 — 전수 검토 대신 '의심 항목 집중 검토'로 효율화. (자기확신-정답률 보정은 확인 권장)

RAG 파이프라인 — 검색으로 근거를 붙이다

개념도(영문 라벨): *information sources* → *chunk* → *vector DB* → *top-k retrieval* → *re-ranking* → *language model* → *answer(+출처)*



+심화

RAG는 6단계 흐름이다: 문서를 조각(chunk)으로 나눠 의미 벡터(embed)로 만들고, 벡터 DB에 저장한 뒤, 질문과 가까운 조각을 top-k로 검색(retrieve)·재순위(rerank)해, 그 근거만 보고 답(generate)한다. '오픈북 시험'과 같다.

RAG — 검색으로 근거를 붙이다

핵심 알고리즘

1. 동기 — LLM 파라미터 지식은 낮고 출처가 없음
2. 검색(Retrieve) — 질문으로 외부 지식(문헌·DB·특허) 검색
3. 증강(Augment) — 검색 결과를 컨텍스트로 프롬프트에 주입
4. 생성(Generate) — 근거에 기반해 답 + 출처 제시
5. 효과 — 환각 완화·최신성·추적가능성(provenance)

차별점 (vs 이전)

- ▶ 출처 있는 답(grounding)
- ▶ 재학습 없이 최신 지식
- ▶ 신약: 논문·실험노트 근거

한계 · 주의

- 검색 품질에 의존
- 근거 밖 추론은 여전히 오답 가능

쉽게 — '찾아보고 답하기'

- 사람도 모르면 '검색·문헌 확인' 후 답한다 — RAG가 그것
- 질문→관련 문서 검색→그 근거만 보고 답(+출처)
- 모델이 지어내는 대신 '실제 문서'에 근거하게 강제
- PaperQA2(FutureHouse): 문헌 QA에서 전문가 수준 주장

쉽게

RAG = '오픈북 시험'. 닫힌 기억(환각)이 아니라 펼친 문서(근거)로 답하게 → 출처가 붙고 사실성이 올라간다. 신약 문헌·특허 검토의 핵심 도구

인덱싱은 한 번, 질의는 매번 — 실제 구현 흐름

1

16.1

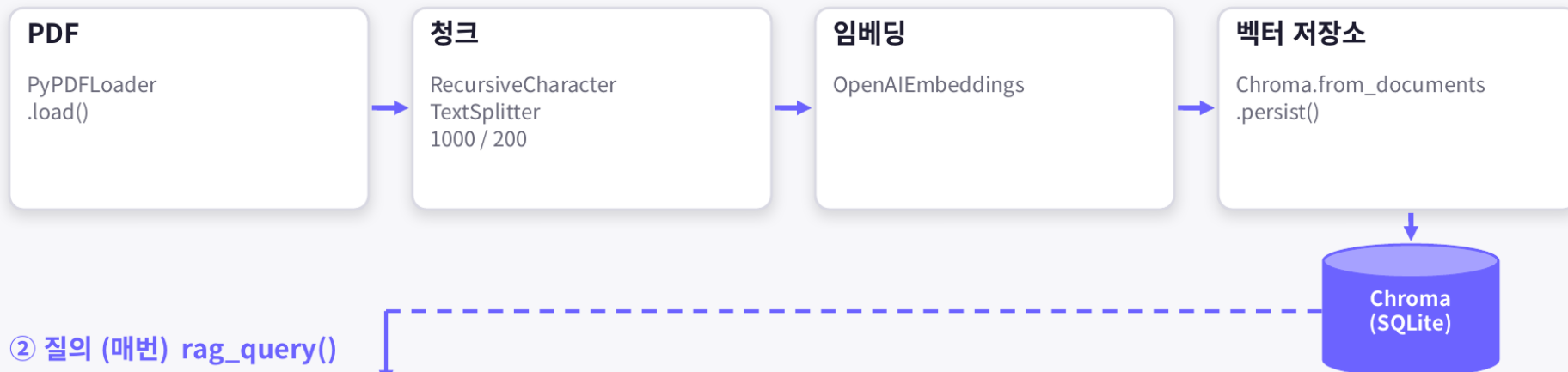
16.2

18

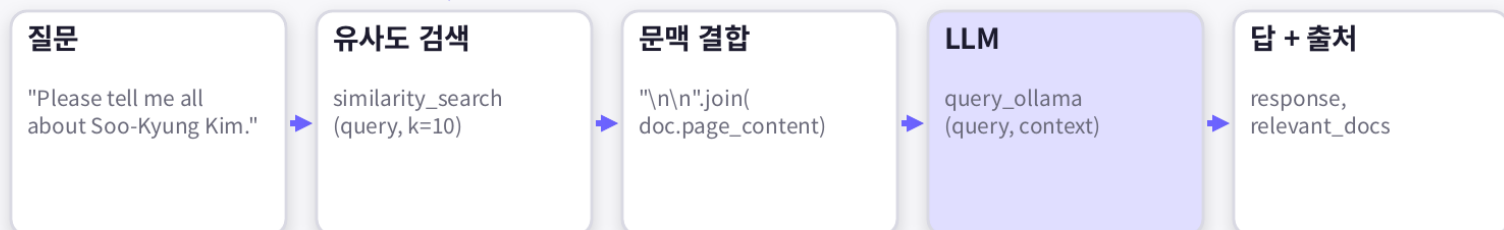
RAG 파이프라인: 인덱싱 → 검색 → 생성

랭체인 + 오픈AI 임베딩 + Chroma + Ollama

① 인덱싱 (한 번)



② 질의 (매번) rag_query()



같은 임베딩으로 질문을 인코딩해야 일치하는 것을 찾을 수 있다 (OpenAIEmbeddings ↔ OpenAIEmbeddings)

▶ ① 인덱싱은 한 번(PDF 로드→청크→임베딩→벡터 저장소), ② 질의는 매번(질문 임베딩→유사도 검색 similarity_search(query, k=10)→문맥 결합→LLM→답+출처). 핵심 규칙: 문서와 질문을 같은 임베딩으로 인코딩해야 매칭된다.

겹침 200자가 하는 일 — 청크 설계가 검색 상한을 정한다

1

16.1

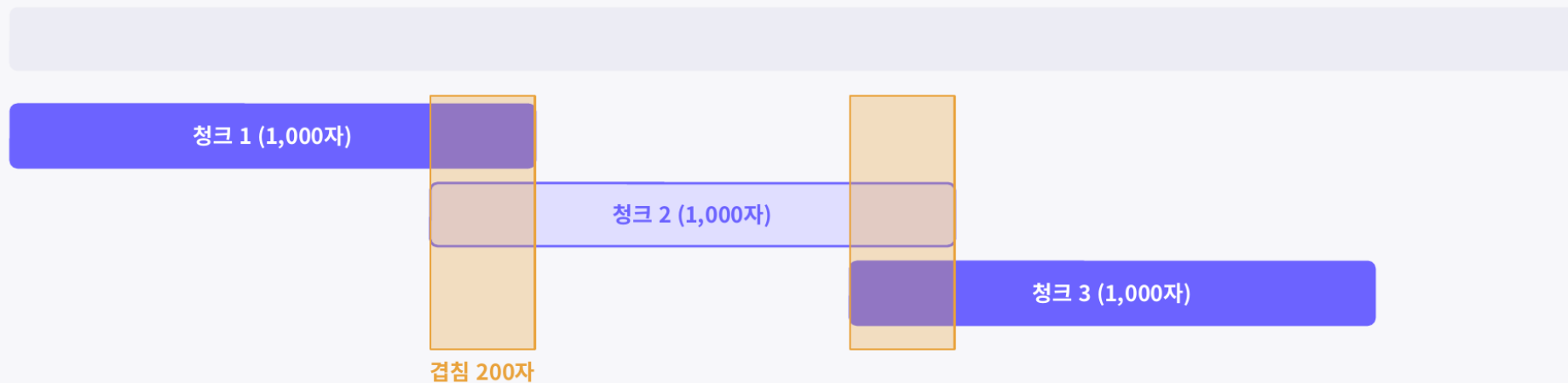
16.2

18

청크 분할: `chunk_size=1000 / overlap=200`

적절한 청크 사이즈를 결정하는 것이 애플리케이션에서 중요한 부분

원문



재귀 전략

정확히 1,000자에서 자르지 않고 자연스러운 경계에서:
줄바꿈 → 문장 → 구두점 → 공백 → (최후) 단어 중간

겹침의 의미

다음 청크가 약 200자 뒤에서 시작. 일부 텍스트가 두 번
들어가지만 문장이 중간에 잘려 내용을 잃지 않는다

크기 트레이드오프

청크가 크면 개수가 적어 검색이 빠르지만, 프롬프트가
청크보다 작으면 매우 유사한 청크를 찾을 가능성이
낮아진다

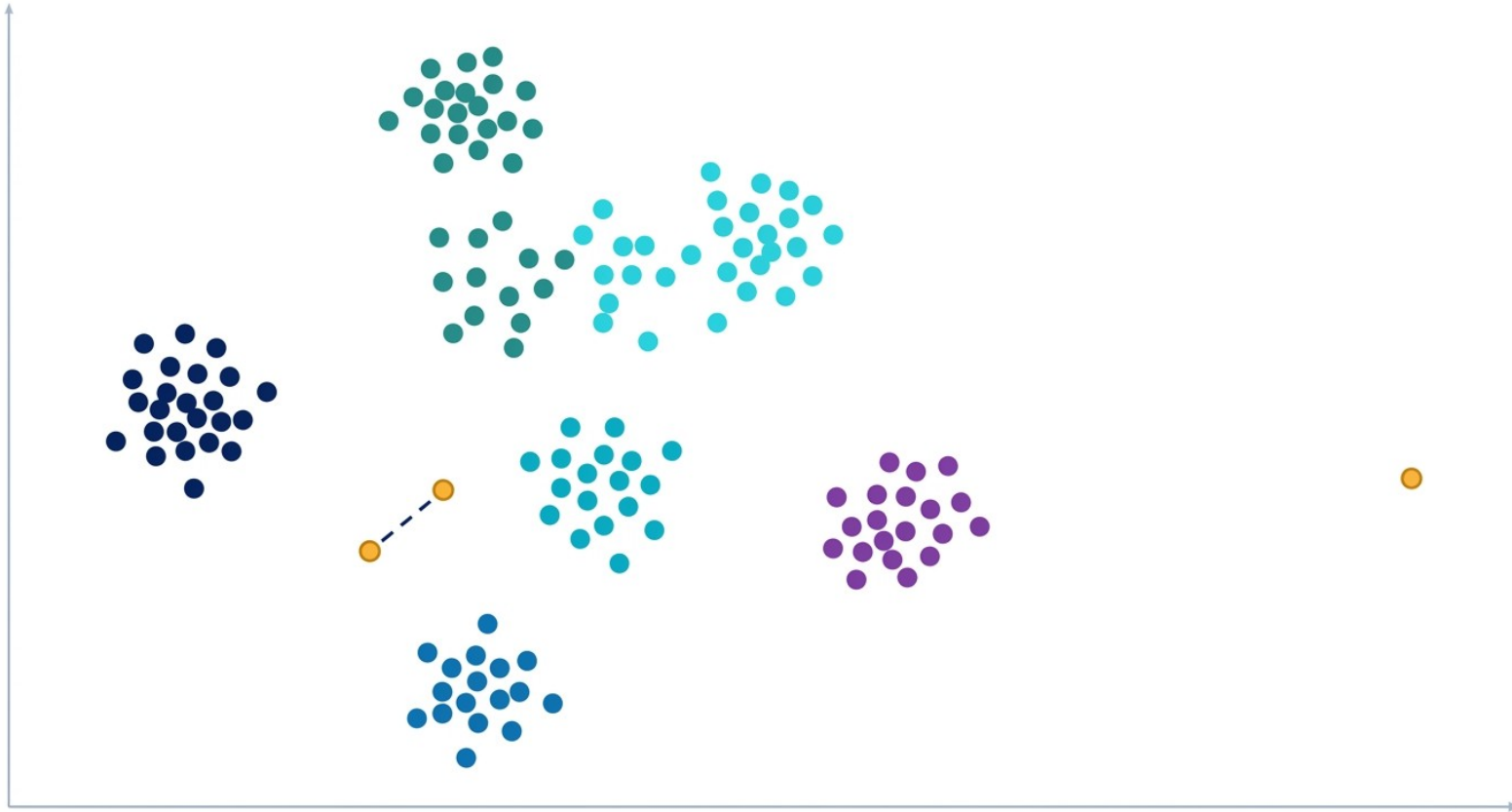
`length_function /`
`add_start_index`

len으로 길이 측정 (모델마다 토큰 셈이 다름: 'lol'=1,
이모지=4). `start_index`는 출처 추적·인용·디버깅용
메타데이터

▶ 재귀 전략은 줄바꿈→문장→구두점→공백 순으로 자연스러운 경계에서 자른다. 겹침 200자는 문장이 중간에 잘려 뜻을 잃는 것을 막는 장치. 청크가 너무 크면 검색은 빠르나 프롬프트가 무겁고, 너무 작으면 유사 청크를 못 찾는다.

임베딩 공간 — 뜻이 가까우면 좌표도 가깝다

무텍스트 개념도: 의미가 비슷한 문장은 군집(cluster), 가까운 두 점은 유사·먼 점은 무관



쉽게

임베딩은 문장을 '의미 좌표(벡터)'로 바꾼다. 뜻이 비슷하면 좌표가 가깝고(같은 색 군집), 다르면 멀다. 그래서 단어가 안 겹쳐도('PDE5 저해제' vs 'PDE5 inhibitor') 의미로 검색할 수 있다.

각도가 작으면 뜻이 가깝다 — 코사인 유사도

1

16.1

16.2

18

유사도 이해하기: 벡터 사이의 각도

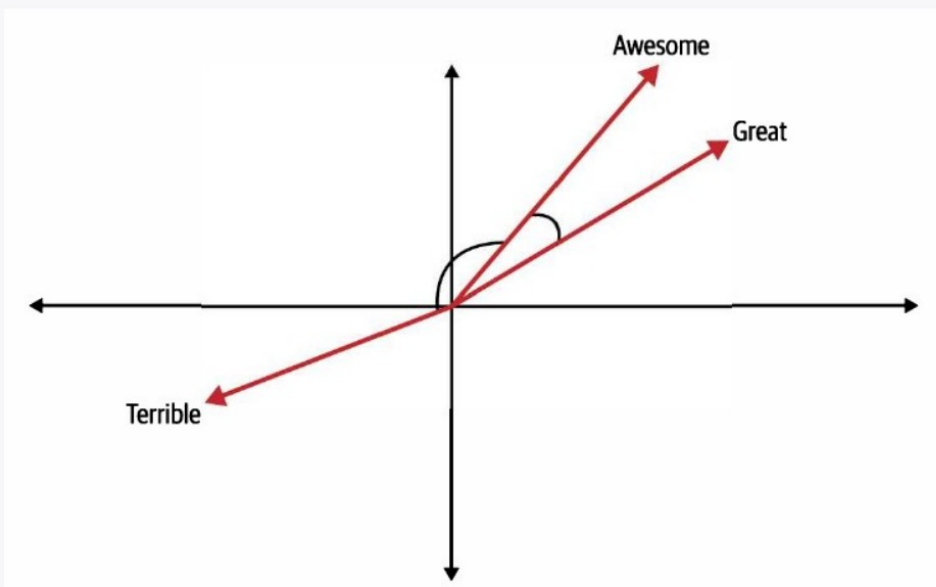


그림 18-4 단어 벡터

코사인은 여러 유사도 계산 방법 중 하나. 튜닝 시 다른 방법도 시도할 것

Awesome ↔ Great

각도 작음 → 유사

Awesome ↔ Terrible

각도 큼 → 유사하지 않음

코사인 유사도로 RAG 구현하기

- 1 책을 청크로 분할
- 2 각 청크의 임베딩 계산
- 3 ChromaDB에 저장
- 4 질문 임베딩과 코사인 유사도가 높은 청크 검색

▶ Awesome↔Great는 각도가 작아 '유사', Awesome↔Terrible은 각도가 커서 '유사하지 않음'. 코사인은 여러 유사도 중 하나일 뿐 — 튜닝 시 다른 척도도 시도할 것. 'PDE5 저해제'와 'PDE5 inhibitor'가 단어는 달라도 가까운 이유가 이것.

의미검색 심화 — dense·sparse·ANN

핵심 알고리즘

1. **sparse(어휘)** — BM25 등 키워드 기반 — 정확 매칭 강함
2. **dense(의미)** — 임베딩 벡터 유사도(cosine) — 동의어 강함
3. **hybrid** — sparse+dense 결합→상호 보완
4. **근접검색(ANN)** — 수백만 벡터를 근사 최근접으로 고속 검색
5. **적용** — 용어 정확성+의미 포괄을 함께 확보

차별점 (vs 이전)

- ▶ 의미+키워드 상호 보완
- ▶ 대규모에서 실시간 검색
- ▶ 도메인 용어에 강건

한계 · 주의

- dense는 색인 비용·갱신 부담
- 임베딩 품질에 의존
- 희귀 용어는 sparse 필요

검색 → 재순위 — 2단계로 정확도 ↑

1차: 빠른 검색(retrieve)

- **bi-encoder**
질문·문서를 각각 벡터화
- **사전계산 색인**
top-k 후보를 빠르게 회수
- **장점**
대규모에서 고속·저비용
- **한계**
정밀 관련도 판단은 약함

2차: 정밀 재순위(rerank)

- **cross-encoder**
질문+문서를 함께 정밀 채점
- **소수 후보만**
top-k만 재평가(비용 절충)
- **효과**
진짜 관련 근거를 상위로
- **결과**
적은 근거로 더 정확한 답

+심화

실무 조합: 값싼 bi-encoder로 넓게 후보를 모으고, 비싼 cross-encoder로 소수만 정밀 재순위. '많이 회수→정밀 선별'이 정확도와 비용을 동시에 잡는 표준 패턴.

RAG 품질 평가 — 무엇을 재나

지표	무엇을 측정 / 주의
검색 recall@k	정답 근거가 top-k에 포함됐나
context precision	검색된 조각이 실제로 관련 있나
faithfulness	답이 근거에서만 나왔나(비약 탐지)
answer relevance	답이 질문에 부합하나
주의	자동지표는 근사 — 사람 검수 병행

+심화

평가 없이는 개선도 없다: 검색이 약한지(recall ↓) 생성이 비약하는지(faithfulness ↓)를 분리 측정해야 어디를 고칠지 안다. 자동 평가 프레임워크(RAGAS 등)는 참고용, 최종은 사람 검수. (세부 지표 정의는 확인 권장)

출처 인용 의무 & 표준 형식

인용 의무화

- 모든 사실 주장에 출처 1개 이상
- 주장 문장 ↔ 근거 문장 매핑
- 근거 없으면 'unknown'/보류
'모르면 모른다'

표준 형식

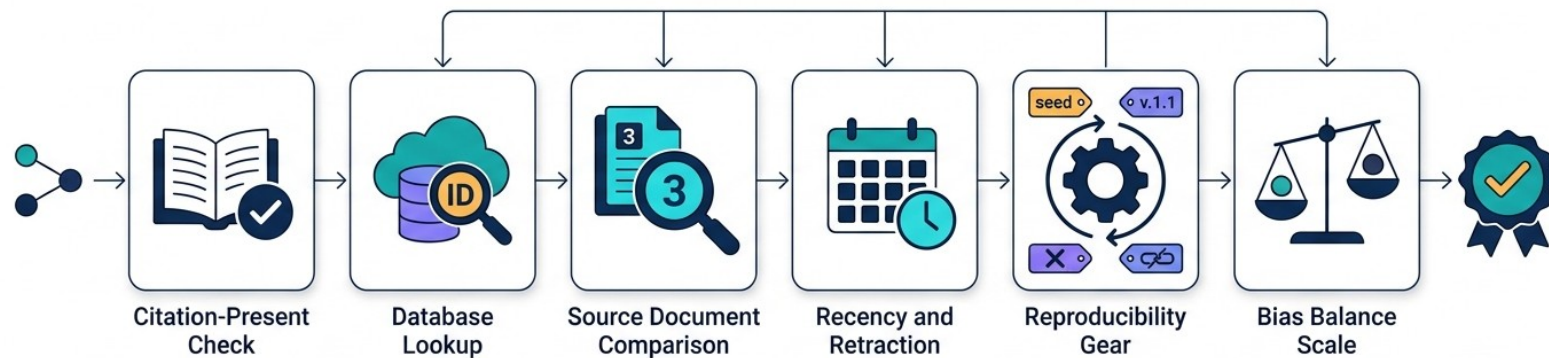
- 논문: 저자 · 연도 · venue · DOI/PMID
- DB: 이름 · 버전 · 접근일 · 쿼리
- 검증 가능한 식별자 필수
가짜 ID 차단

+심화

핵심 규범: (1) 문장마다 근거를 붙이고, (2) 그 식별자(DOI/PMID)가 실재하는지 확인. 본 과정에서도 '레퍼런스는 기본' — 인용 표준을 습관화해야 환각을 걸러낸다.

사실성 검증 워크플로우 — 6개 게이트

개념도(영문 라벨): 인용 존재(citation check) → 식별자 실재(DB lookup) → 원문 대조(source comparison) → 최신·철회(recency) → 재현성(reproducibility) → 편향(bias) → 검증 배지



+심화

AI 답은 '가설'이다. 6개 게이트를 순서대로 통과해야 인용·의사결정에 쓴다: ① 출처 있나 → ② 식별자(DOI/PMID) 실재하나 → ③ 수치가 원문과 같나 → ④ 최신·철회 확인 → ⑤ 재현 가능(seed·버전) → ⑥ 편향 점검. ②가 가장 강력한 필터.

사실성 검증 체크리스트 (RAG 교차확인)

점검 항목	확인 방법
① 출처가 명시됐나	주장마다 인용 존재
② 출처가 실재하나	DOI/PMID 실제 조회(가짜 ID 차단)
③ 수치·주장이 원문과 일치하나	원문 대조(요약 왜곡 점검)
④ 최신·철회 여부	발행일 · 정정/철회 확인
⑤ 재현 가능한가	프롬프트 · 모델버전 · seed 기록
⑥ 이해상충 · 편향	출처 편중 · 상업적 편향 점검

+심화

실무 규칙: LLM 답은 '가설'로 취급 → 이 6항목을 통과해야 인용·의사결정에 사용. ②(출처 실재 조회)가 가장 강력한 환각 필터.

사실성 검증 — 코드로 거르기(개념)

인용 실재성 자동 점검 (② 게이트)

```
for claim in answer.claims:          # 답의 각 주장
    if not claim.citation:
        flag(claim, "NO_SOURCE")      # ① 출처 없음
    elif not exists_in(claim.pmid, PubMed):
        flag(claim, "FAKE_ID")        # ② 가짜 DOI/PMID
    elif not match(claim.value, source.text):
        flag(claim, "MISMATCH")       # ③ 원문 불일치
```

▶ 식별자 실재 조회·원문 수치 대조는 자동화 가능 — 사람은 flag된 항목만 집중 검토(효율 ↑)

+심화

체크리스트 6항목 중 ①②③은 코드로 상당 부분 자동화된다(존재 조회·문자열/수치 대조). ④~⑥과 최종 판단은 사람 몫. '자동 필터 + 사람 심판'이 재직자 실무의 효율적 검증 구조.

RAG도 만능은 아니다

- 검색이 틀리면(누락·부정확) 답도 틀림 — 검색 품질이 상한
- 근거 밖으로 '추론'하면 여전히 환각
- 긴 문맥·다중 문서 종합에서 왜곡 가능
- → 인간 검증·교차확인 여전히 필수



함정

RAG가 붙었다고 맹신 금지 — 검색이 놓치거나 모델이 근거를 넘어 추론하면 그럴듯한 오답이 나온다. 최종 판단·재현은 사람 몫

RAG 실패모드 — 어디서 무너지나

실패 지점	증상 / 대응
검색 누락	관련 근거를 못 찾음 → hybrid·rerank·chunk 개선
잡음 근거	무관 조각 주입 → precision·rerank 강화
근거 무시	근거 있어도 파라미터 지식으로 답 → 'only from context' 지시
근거 밖 비약	근거 넘어 추론 → faithfulness 검사·인용 강제
색인 노후	오래된 정보 → 주기적 재색인

+심화

디버깅 순서: 먼저 '검색이 근거를 가져왔나'(retrieve)를 보고, 가져왔다면 '답이 근거만 썼나'(faithfulness)를 본다. 이 둘을 분리해야 chunk·rerank·프롬프트 중 무엇을 고칠지 안다.

실전 — LLM+RAG 문헌 리뷰 (예시)

① 질문 정의

'타겟 X 저해제 계열·근거'를 구조화 질의

② 검색(RAG)

PubMed·특허 색인에서 관련 문헌 top-k

③ 근거 추출

각 주장에 PMID/DOI 매핑·표 생성

④ 검증

출처 실재 조회·원문 수치 대조(체크리스트)

⑤ 산출

요약+출처표+불확실 항목 플래그

+심화

이 흐름이 3교시 에이전트의 원형 — 여기서 사람이 각 단계를 지시하지만, 3교시에선 에이전트가 ①~⑤를 스스로 오케스트레이션한다.

실제 문헌-QA 시스템 — 어디까지 왔나

- PaperQA2(FutureHouse): 전문 문헌 RAG QA — 일부 과제서 전문가 수준 주장
- 범용 LLM + PubMed/특허 커넥터 + RAG로 사내 조합 가능
- 공통 설계: 검색→근거 추출→인용→자기검증 루프
- 주의: '주장'은 벤치·조건 의존 — 자기 데이터로 재검증 권장

+심화

교훈: 문헌 리뷰 자동화는 이미 실용 단계에 진입. 다만 성능 주장은 특정 벤치·설정에서의 결과이므로, 도입 시 우리 문헌·질문으로 다시 평가하고 인용을 검증하는 절차가 필수(확인 권장).

PART C

정리

실무 원칙 · 벤치마크 함정 · 3교시 예고

재직자 실무 원칙 — DO & DON'T

DO (권장)

- 출처·근거를 항상 요구
- RAG로 사내 문헌·노트에 grounding
- 구조화 출력→자동 검증 파이프라인
- 답은 '가설'로 취급·교차확인

DON'T (금지)

- 검증 없이 결과 인용·의사결정
- LLM 단독으로 분자 생성 맹신
- 민감데이터 무단 입력
- 가짜 DOI/PMID 미확인 방치

쉽게

한 줄 요약: '근거를 붙이게 하고, 사람이 검증한다.' 이 두 가지만 지켜도 현업에서 LLM을 안전하게 쓸 수 있다.

한 장 요약 — 활용→검증 파이프라인

단계	핵심 행동 / 산출
① 설계	역할·지시·few-shot·구조화 출력·근거 요구
② 추론 강화	CoT·분해·self-consistency로 안정화
③ 근거 부착	RAG(chunk→embed→retrieve→rerank)
④ 검증	6항목 체크리스트·출처 실재 조회
⑤ 기록	프롬프트·모델버전·seed·출처 로그(재현)

+심화

이 5단계가 2교시의 결론 — 프롬프트로 '잘 묻고', RAG로 '근거를 붙이고', 체크리스트로 '검증하고', 로그로 '재현'한다. 3교시는 이 흐름을 에이전트가 자동 수행한다.

LLM · 과학 역량 벤치마크

벤치마크	내용
LAB-Bench	2,400+ 생물 연구역량(문헌 · 프로토콜 · DB · 그림)
BixBench	생명정보학 에이전트 평가
MedQA/PubMedQA	의료 · 생의학 QA
주의	벤치 오염 · 실제 태스크와 격차 → 맹신 금지

쉽게

평가도 진화 중 — 단순 지식 QA에서 '문헌 검색 · 프로토콜 이해 · 데이터 분석' 같은 실제 연구 역량(LAB-Bench)으로. 단 점수는 참고일 뿐, 현업 검증이 우선

벤치마크 점수를 믿기 전에

- 데이터 오염(contamination): 테스트가 학습에 섞이면 점수 부풀림
- 벤치-현업 격차: 시험 잘 봐도 우리 데이터·질문엔 다를 수 있음
- 지표 편향: 단일 점수는 실패 유형(환각·비약)을 감춤
- → 도입 결정은 '자기 태스크'로 재평가 후 (LAB-Bench는 방향 참고)



함정

'SOTA 점수'는 마케팅이 되기 쉽다. 재직자 판단 기준: 우리 문헌·우리 질문으로 소규모 파일럿을 돌려 환각률·인용 정확도를 직접 측정하라.

2교시 정리

1

범용 LLM = 문헌·데이터·가설·코드의 만능 조수

2

프롬프트: 역할·지시·few-shot·CoT·분해·self-consistency·구조화 출력·출처 요구

3

API·함수호출·구조화 출력로 파이프라인화(→에이전트 토대)

4

환각은 원리적(확률적 이어쓰기) — 4층(프롬프트·RAG·양상블·검증)으로 완화

5

RAG(chunk→embed→retrieve→rerank) + 인용 표준 + 6항목 체크리스트로 검증

6

AI 답은 '가설' — 검증·재현은 사람 몫

다음: 에이전트·보안

3교시 예고

스스로 도구를 쓰게 — LLM 에이전트

ReAct · tool use · 계획 · 메모리 · ChemCrow · Coscientist · AI co-scientist · Robin · Claude for LS · DMTA 자율화

프롬프트 · RAG를 넘어, 도구를 '스스로 쓰는' 에이전트로. (보안 · 엔지니어링은 4교시)

주요 출처 (2교시)

- 프롬프트·활용: Wei et al. 2022 (CoT, 2201.11903); Wang et al. 2023 (Self-Consistency, ICLR, 2203.11171); Madaan et al. 2023 (Self-Refine); Brown et al. 2020 (GPT-3 in-context, 2005.14165); Liu et al. 2023 (Prompting Survey); Schick et al. 2023 (Toolformer).
- RAG·검색: Lewis et al. 2020 (RAG, NeurIPS, 2005.11401); Karpukhin et al. 2020 (DPR); Reimers & Gurevych 2019 (Sentence-BERT); Gao et al. 2023 (RAG survey); Nogueira & Cho 2019 (BERT re-ranking, 확인 권장).
- 환각: Ji et al. 2023 (Hallucination Survey, ACM Comput Surv); Taylor et al. 2022 (Galactica, 2211.09085); Liu et al. 2023 (Lost in the Middle).
- 문헌-QA·벤치마크: Skarlinski et al. 2024 (PaperQA2, 2409.13740); Laurent et al. 2024 (LAB-Bench, 2407.10362); FutureHouse 2025 (BixBench, 확인 권장).
- 화학 LLM 활용: Bran et al. 2024 (ChemCrow); Ye et al. 2024 (DrugAssist).
- * 인용 표준·검증 체크리스트·RAG 평가(RAGAS 등)는 과학 저술 규범 + 본 강의 QA 원칙 기반. 일부 서지·수치는 확인 권장 표기.